

```
In [1]: import pandas as pd
import os
from sqlalchemy import create_engine
import sqlite3
import mysql.connector as connection
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [ ]: engine = create_engine('sqlite:///Healthcare_Analytics.db')
```

```
In [3]: for file in os.listdir("C:\\Users\\Lenovo\\Downloads\\Healthcare _Analytics project 12"):
print(file)
```

```
.ipynb_checkpoints
Billing.csv
Cleaned_Appointment.csv
Cleaned_Doctor.csv
Cleaned_Patient.csv
folder
Medical Procedure (1).csv
```

```
In [5]: for file in os.listdir("C:\\Users\\Lenovo\\Downloads\\Healthcare _Analytics project 12"):
if '.csv' in file:
df = pd.read_csv("C:\\Users\\Lenovo\\Downloads\\Healthcare _Analytics project 12/"+file)
print(df.shape)
ingest_db(df,file[:-4],engine)
```

```
(1000, 4)
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[5], line 5
      1 for file in os.listdir("C:\\Users\\Lenovo\\Downloads\\Healthcare _Analytics project 12"):
      2     if '.csv' in file:
      3         df = pd.read_csv("C:\\Users\\Lenovo\\Downloads\\Healthcare _Analytics project 12/"+file)
      4         print(df.shape)
----> 5         ingest_db(df,file[:-4],engine)

NameError: name 'ingest_db' is not defined
```

```
In [6]: def ingest_db(df,table_name,engine):
df.to_sql(table_name,con=engine,if_exists='replace',index=False)
```

<h1 id="Exploratory-Data-Analysis">Exploratory Data Analysis</h1>

```
In [ ]:
```

```
In [ ]: # creating database connection
```

```
In [7]: conn= sqlite3.connect('Healthcare_Analytics.db')
```

```
In [ ]: # checking table present inthe database
```

```
In [8]: tables= pd.read_sql_query("SELECT name from Sqlite_master WHERE type='table'",conn)
tables
```

```
Out[8]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top;
} .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>name</th> </thead> <tbody>
<th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> </tbody> <tbody>
</tbody>
```

doctor

Patient

Billing

Cleaned_Appointment

Cleaned_Doctor

Cleaned_Patient

Medical Procedure (1)

```
In [ ]:
```

```
In [9]: pd.read_sql("select count(*) from Cleaned_Patient",conn)
```

```
Out[9]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top;
} .dataframe tthead th { text-align: right; } </style> <tthead> <th></th> <th>count(<*></th> </thead> <tbody>
<th>0</th> </tbody> <tbody></tbody>
1000
```

```
In [10]: for table in tables['name'] :
print('-'*50,f'{table}','-'*50)
print('Count of records:',pd.read_sql(f"select count(*) as count from {table}",conn)['count'].values[0])
display(pd.read_sql(f"SELECT * FROM {table} limit 5",conn))
```

----- doctor -----
Count of records: 10
<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; }
.dataframe tthead th { text-align: right; } </style> <tthead> <th></th> <th>DoctorID</th> <th>DoctorName</th>
<th>Specialization</th> <th>DoctorContact</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th>
<th>3</th> <th>4</th> </tbody> <tbody></tbody>

116	Shell	Infectious disease	.@yopmail.com
752	Regina	Oncologist	.@yopmail.com
957	Sissy	Otolaryngologists	.@yopmail.com
217	Celestyna	Cardiology	.@yopmail.com
100	Thalia	Emergency Medicine	.@yopmail.com

----- Patient -----
Count of records: 10
<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; }
.dataframe tthead th { text-align: right; } </style> <tthead> <th></th> <th>PatientID</th> <th>FirstName</th>
<th>LastName</th> <th>Email</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th>
<th>4</th> </tbody> <tbody></tbody>

919	Anallese	Halla	Anallese.Halla@yopmail.com
535	Shel	Dearborn	Shel.Dearborn@yopmail.com
830	Mara	Shuler	Mara.Shuler@yopmail.com
398	Dale	Lewes	Dale.Lewes@yopmail.com
375	Sharlene	Luhe	Sharlene.Luhe@yopmail.com

----- Billing -----
Count of records: 1000
<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; }
.dataframe tthead th { text-align: right; } </style> <tthead> <th></th> <th>InvoiceID</th> <th>PatientID</th>
<th>Items</th> <th>Amount</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th>
<th>4</th> </tbody> <tbody></tbody>

e9609dce-f125-4f65-a067-4760a06e60b6	894	Immunizations	956065
fb593fb6-466a-4dc8-9a61-afbd9d4b5031	448	Cataract surgery	188997
6fecc813-a9aa-4a89-97b5-079f3842eeeb	618	Pediatric surgery	524091
09bb8e2f-4fe0-45c4-8963-d3ccc4c96e5c	117	Rhinoplasty	901393
5fba7427-10ac-423f-a6d1-d815710e17c4	374	Antibiotic therapy	448957

----- Cleaned_Appointment -----
Count of records: 1000
<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; }
.dataframe tthead th { text-align: right; } </style> <tthead> <th></th> <th>AppointmentID</th> <th>Date</th>
<th>Time</th> <th>PatientID</th> <th>DoctorID</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th>
<th>3</th> <th>4</th> </tbody> <tbody></tbody>

639	2022-04-08	2023-12-23T14:33:46.406Z	109	462
404	2023-04-10	2023-12-23T14:33:46.407Z	823	774
281	2022-12-09	2023-12-23T14:33:46.407Z	465	226
628	2020-01-08	2023-12-23T14:33:46.407Z	443	656
889	2023-08-30	2023-12-23T14:33:46.407Z	852	262

----- Cleaned_Doctor -----
Count of records: 1000

```
<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; }
.dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>DoctorID</th> <th>DoctorName</th>
<th>Specialization</th> <th>DoctorContact</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th>
<th>3</th> <th>4</th> </tbody> <tbody></tbody>

116      Shell      Infectious disease  .@yopmail.com
752      Regina      Oncologist  .@yopmail.com
957      Sissy      Otolaryngologists  .@yopmail.com
217      Celestyna      Cardiology  .@yopmail.com
100      Thalia      Emergency Medicine  .@yopmail.com
```

----- Cleaned_Patient -----

```
-----
Count of records: 1000
<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; }
.dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>PatientID</th> <th>FirstName</th>
<th>LastName</th> <th>Email</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th>
<th>4</th> </tbody> <tbody></tbody>

919  Anallese      Halla  Anallese.Halla@yopmail.com
535   Shel  Dearborn  Shel.Dearborn@yopmail.com
830   Mara   Shuler   Mara.Shuler@yopmail.com
398   Dale   Lewes   Dale.Lewes@yopmail.com
375  Sharlene   Luhe   Sharlene.Luhe@yopmail.com
```

----- Medical Procedure (1) -----

```

-----
OperationalError                                Traceback (most recent call last)
File ~\AppData\Roaming\Python\Python314\site-packages\pandas\io\sql.py:2745, in SQLiteDatabase.execute(self, sql
, params)
    2744 try:
-> 2745     cur.execute(sql, *args)
    2746     return cur

```

OperationalError: near "(": syntax error

The above exception was the direct cause of the following exception:

```

DatabaseError                                Traceback (most recent call last)
Cell In[10], line 3
      1 for table in tables['name'] :
      2     print('-'*50,f'{table}','-'*50)
----> 3     print('Count of records:',pd.read_sql(f"select count(*) as count from {table}",conn)['count'].values
[0])
      4     display(pd.read_sql(f"SELECT * FROM {table} limit 5",conn))

```

```

File ~\AppData\Roaming\Python\Python314\site-packages\pandas\io\sql.py:702, in read_sql(sql, con, index_col, coe
rce_float, params, parse_dates, columns, chunksize, dtype_backend, dtype)
    700 with pandasSQL_builder(con) as pandas_sql:
    701     if isinstance(pandas_sql, SQLiteDatabase):
-> 702         return pandas_sql.read_query(
    703             sql,
    704             index_col=index_col,
    705             params=params,
    706             coerce_float=coerce_float,
    707             parse_dates=parse_dates,
    708             chunksize=chunksize,
    709             dtype_backend=dtype_backend,
    710             dtype=dtype,
    711         )
    713     try:
    714         _is_table_name = pandas_sql.has_table(sql)

```

```

File ~\AppData\Roaming\Python\Python314\site-packages\pandas\io\sql.py:2809, in SQLiteDatabase.read_query(self,
sql, index_col, coerce_float, parse_dates, params, chunksize, dtype, dtype_backend)
    2798 def read_query(
    2799     self,
    2800     sql,
    (...) 2807     dtype_backend: DtypeBackend | Literal["numpy"] = "numpy",
    2808 ) -> DataFrame | Iterator[DataFrame]:
-> 2809     cursor = self.execute(sql, params)
    2810     columns = [col_desc[0] for col_desc in cursor.description]
    2812     if chunksize is not None:

```

```

File ~\AppData\Roaming\Python\Python314\site-packages\pandas\io\sql.py:2809, in SQLiteDatabase.read_query(self,
sql, index_col, coerce_float, parse_dates, params, chunksize, dtype, dtype_backend)
    2798 def read_query(
    2799     self,
    2800     sql,
    (...) 2807     dtype_backend: DtypeBackend | Literal["numpy"] = "numpy",
    2808 ) -> DataFrame | Iterator[DataFrame]:
-> 2809     cursor = self.execute(sql, params)
    2810     columns = [col_desc[0] for col_desc in cursor.description]
    2812     if chunksize is not None:

```

```

    2798 def read_query(
    2799     self,
    2800     sql,
    (...) 2807     dtype_backend: DtypeBackend | Literal["numpy"] = "numpy",
    2808 ) -> DataFrame | Iterator[DataFrame]:
-> 2809     cursor = self.execute(sql, params)
    2810     columns = [col_desc[0] for col_desc in cursor.description]
    2812     if chunksize is not None:

```

```

File ~\AppData\Roaming\Python\Python314\site-packages\pandas\io\sql.py:2757, in SQLiteDatabase.execute(self, sql
, params)
    2754     raise ex from inner_exc
    2756 ex = DatabaseError(f"Execution failed on sql '{sql}': {exc}")
-> 2757     raise ex from exc

```

DatabaseError: Execution failed on sql 'select count(*) as count from Medical Procedure (1)': near "(": syntax error

```

In [12]: Patient = pd.read_sql_query(" select * from Cleaned_Patient LIMIT 10",conn)
Patient

```

```

Out[12]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top;
} .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>PatientID</th> <th>FirstName</th>
<th>LastName</th> <th>Email</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th>
<th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> </tbody> <tbody></tbody>

```

919	Anallese	Halla	Anallese.Halla@yopmail.com
-----	----------	-------	----------------------------

535	Shel	Dearborn	Shel.Dearborn@yopmail.com
-----	------	----------	---------------------------

830	Mara	Shuler	Mara.Shuler@yopmail.com
-----	------	--------	-------------------------

398	Dale	Lewes	Dale.Lewes@yopmail.com
-----	------	-------	------------------------

375	Sharlene	Luhe	Sharlene.Luhe@yopmail.com
-----	----------	------	---------------------------

566	Rebeca	Winnick	Rebeca.Winnick@yopmail.com
-----	--------	---------	----------------------------

404	Merry	Henebry	Merry.Henebry@yopmail.com
-----	-------	---------	---------------------------

922	Orsola	Torray	Orsola.Torray@yopmail.com
-----	--------	--------	---------------------------

603	Dolli	Anderea	Dolli.Anderea@yopmail.com
-----	-------	---------	---------------------------

939	Mellicent	Shama	Mellicent.Shama@yopmail.com
-----	-----------	-------	-----------------------------

In []:

In [13]: Billings = pd.read_sql_query("select * from Billing LIMIT 10",conn)
Billings

Out[13]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>InvoiceID</th> <th>PatientID</th> <th>Items</th> <th>Amount</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> </tbody> <tbody></tbody>

e9609dce-f125-4f65-a067-4760a06e60b6	894	Immunizations	956065
fb593fb6-466a-4dc8-9a61-afbd9d4b5031	448	Cataract surgery	188997
6fecc813-a9aa-4a89-97b5-079f3842eeeb	618	Pediatric surgery	524091
09bb8e2f-4fe0-45c4-8963-d3ccc4c96e5c	117	Rhinoplasty	901393
5fba7427-10ac-423f-a6d1-d815710e17c4	374	Antibiotic therapy	448957
d17275f1-fd48-4dc7-8c44-8425ef0d38ea	852	Emergency surgical procedures	869041
7cebd89b-9329-4722-84c3-3a2ab3430277	834	Pediatric surgery	492022
921c7c7d-c2c7-44b9-b9c6-6a325f1596fd	140	Hemodynamic monitoring	644410
43acf5cc-57d8-4a23-ba2e-42481d1189d6	459	Emergency surgical procedures	295907
b134ec92-b0ce-451a-9f5f-8bfbf96cc02f	734	Immunotherapy (allergy shots)	766908

In [43]: Doctor = pd.read_sql_query("select * from Cleaned_Doctor LIMIT 10 ",conn)
Doctor

Out[43]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>DoctorID</th> <th>DoctorName</th> <th>Specialization</th> <th>DoctorContact</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> </tbody> <tbody></tbody>

116	Shell	Infectious disease	.@yopmail.com
752	Regina	Oncologist	.@yopmail.com
957	Sissy	Otolaryngologists	.@yopmail.com
217	Celestyna	Cardiology	.@yopmail.com
100	Thalia	Emergency Medicine	.@yopmail.com
708	Brynna	Surgery	.@yopmail.com
262	Maye	Pulmonologists	.@yopmail.com
378	Concettina	Hospice and Palliative Care	.@yopmail.com
910	Minda	Anesthesiology	.@yopmail.com
740	Charissa	Nephrology	.@yopmail.com

In [79]: Patient.head()
Doctor.head()
Billings.head()
Appointment.head()

Out[79]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>AppointmentID</th> <th>Date</th> <th>Time</th> <th>PatientID</th> <th>DoctorID</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

639	2022-04-08	2023-12-23T14:33:46.406Z	109	462
404	2023-04-10	2023-12-23T14:33:46.407Z	823	774
281	2022-12-09	2023-12-23T14:33:46.407Z	465	226
628	2020-01-08	2023-12-23T14:33:46.407Z	443	656
889	2023-08-30	2023-12-23T14:33:46.407Z	852	262

In []: #Understand the Data

In [80]: Patient.info()
Doctor.info()

```
Billings.info()
Appointment.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  -
0   PatientID   10 non-null    int64
1   FirstName   10 non-null    str
2   LastName    10 non-null    str
3   Email       10 non-null    str
dtypes: int64(1), str(3)
memory usage: 452.0 bytes
<class 'pandas.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  -
0   DoctorID     10 non-null    int64
1   DoctorName    10 non-null    str
2   Specialization 10 non-null    str
3   DoctorContact 10 non-null    str
dtypes: int64(1), str(3)
memory usage: 452.0 bytes
<class 'pandas.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  -
0   InvoiceID    10 non-null    str
1   PatientID    10 non-null    int64
2   Items        10 non-null    str
3   Amount       10 non-null    int64
dtypes: int64(2), str(2)
memory usage: 452.0 bytes
<class 'pandas.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 5 columns):
#   Column      Non-Null Count  Dtype
---  -
0   AppointmentID 10 non-null    int64
1   Date          10 non-null    str
2   Time          10 non-null    str
3   PatientID     10 non-null    int64
4   DoctorID      10 non-null    int64
dtypes: int64(3), str(2)
memory usage: 532.0 bytes
```

```
In [81]: Patient.shape
Doctor.shape
Billings.shape
Appointment.shape
```

```
Out[81]: (10, 5)
```

```
In [82]: Patient.describe()
Billings.describe()
```

```
Out[82]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top;
} .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>PatientID</th> <th>Amount</th>
</thead> <tbody> <th>count</th> <th>mean</th> <th>std</th> <th>min</th> <th>25%</th> <th>50%</th>
<th>75%</th> <th>max</th> </tbody> <tbody></tbody>
```

```
10.000000    10.000000
```

```
547.000000    608779.100000
```

```
285.547427    262613.852182
```

```
117.000000    188997.000000
```

```
392.500000    459723.250000
```

```
538.500000    584250.500000
```

```
809.000000    843507.750000
```

```
894.000000    956065.000000
```

<h1 id="Data-Quality-/Cleaning">Data Quality / Cleaning</h1>

```
In [ ]: #Check missing values
```

```
In [83]: print(Patient.isnull().sum())
print(Doctor.isnull().sum())
print(Billings.isnull().sum())
print(Appointment.isnull().sum())
```

```
PatientID      0
FirstName      0
LastName       0
Email          0
dtype: int64
DoctorID       0
DoctorName     0
Specialization 0
DoctorContact  0
dtype: int64
InvoiceID      0
PatientID      0
Items          0
Amount         0
dtype: int64
AppointmentID  0
Date           0
Time           0
PatientID      0
DoctorID       0
dtype: int64
```

<h1 id="Check-duplicates">Check duplicates</h1>

```
In [84]: print(Patient.duplicated().sum())
print(Doctor.duplicated().sum())
print(Billings.duplicated().sum())
print(Appointment.duplicated().sum())
```

```
0
0
0
0
```

```
In [80]: Appointment= pd . read_sql_query("SELECT * FROM Cleaned_Appointment LIMIT 10",conn)
Appointment
```

```
Out[80]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top;
} .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>AppointmentID</th> <th>Date</th>
<th>Time</th> <th>PatientID</th> <th>DoctorID</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th>
<th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> </tbody> <tbody>
</tbody>
```

639	2022-04-08	2023-12-23T14:33:46.406Z	109	462
404	2023-04-10	2023-12-23T14:33:46.407Z	823	774
281	2022-12-09	2023-12-23T14:33:46.407Z	465	226
628	2020-01-08	2023-12-23T14:33:46.407Z	443	656
889	2023-08-30	2023-12-23T14:33:46.407Z	852	262
551	2022-07-04	2023-12-23T14:33:46.407Z	160	748
769	2020-11-24	2023-12-23T14:33:46.407Z	886	580
385	2022-02-26	2023-12-23T14:33:46.407Z	480	438
813	2021-01-24	2023-12-23T14:33:46.407Z	634	662
986	2022-05-09	2023-12-23T14:33:46.407Z	898	653

```
In [ ]: #Important Validation
# Find appointments with invalid doctors
```

```
In [70]: Invalid = pd.read_sql_query(""" select a.* from Cleaned_Appointment A
left join Cleaned_Doctor D
on A.DoctorID = D.DoctorID
where D .DoctorID IS NULL""",conn)
```

```
In [71]: Invalid
```

```
Out[71]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top;
} .dataframe tthead th { text-align: right; } </style> <tthead> <th></th> <th>AppointmentID</th> <th>Date</th>
<th>Time</th> <th>PatientID</th> <th>DoctorID</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th>
<th>3</th> <th>4</th> <th>...</th> <th>321</th> <th>322</th> <th>323</th> <th>324</th> <th>325</th>
</tbody></tbody>

639  2022-04-08  2023-12-23T14:33:46.406Z  109  462
404  2023-04-10  2023-12-23T14:33:46.407Z  823  774
986  2022-05-09  2023-12-23T14:33:46.407Z  898  653
778  2020-01-15  2023-12-23T14:33:46.407Z  948  441
834  2022-09-01  2023-12-23T14:33:46.407Z  104  390
...      ...      ...      ...      ...
340  2021-06-15  2023-12-23T14:33:46.417Z  680  313
585  2023-05-30  2023-12-23T14:33:46.417Z  595  698
772  2021-09-15  2023-12-23T14:33:46.417Z  461  833
427  2021-02-06  2023-12-23T14:33:46.417Z  864  698
205  2020-12-24  2023-12-23T14:33:46.417Z  968  793

<p>326 rows × 5 columns</p>
```

```
In [ ]:
```

```
In [72]: Invalid_Patients = pd.read_sql_query(""" select B.*from  Billing B
left join Patient P
on B.PatientID = P.PatientID
where P.PatientID IS NULL """,conn)
```

```
In [73]: Invalid_Patients
```

```
Out[73]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top;
} .dataframe tthead th { text-align: right; } </style> <tthead> <th></th> <th>InvoiceID</th> <th>PatientID</th>
<th>Items</th> <th>Amount</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th>
<th>4</th> <th>...</th> <th>979</th> <th>980</th> <th>981</th> <th>982</th> <th>983</th> </tbody>
<tbody></tbody>

e9609dce-f125-4f65-a067-4760a06e60b6  894      Immunizations  956065
fb593fb6-466a-4dc8-9a61-afbd9d4b5031  448      Cataract surgery  188997
6fecc813-a9aa-4a89-97b5-079f3842eeeb  618      Pediatric surgery  524091
09bb8e2f-4fe0-45c4-8963-d3ccc4c96e5c  117      Rhinoplasty  901393
5fba7427-10ac-423f-a6d1-d815710e17c4  374      Antibiotic therapy  448957
...      ...      ...      ...
53234ea7-c865-4ec6-a5ff-fb60af896083  365      General anesthesia for surgeries  457946
cdb62ede-7964-4ddd-a605-6fb9bb021702  651      Pacemaker implantation  702692
ed89afa0-01c6-4a00-ab7a-5ab5f559017d  434      Radiation therapy  914950
52adc282-dae6-4099-98f4-86cd1d5702ff  883      Surgical oncology  218009
a48f68fc-f2fb-490f-8327-4ba51300e674  112      Chronic disease management  975444

<p>984 rows × 4 columns</p>
```

```
In [ ]:
```

```
In [ ]: # Specialization Demand
```

```
In [ ]: #Q1. Which doctor is handling the highest number of appointments?
```

```
In [109]: Doctor_Analysis = pd.read_sql_query(""" select D.DoctorName,D .Specialization,
count(A.AppointmentID) AS TotalAppointments
from Cleaned_Doctor D
left join  Cleaned_Appointment A on  D.DoctorID =  A.DoctorID
group by
D.DoctorID ,
D.DoctorName,
D.Specialization
order by TotalAppointments DESC""",conn)
```

```
In [110]: Doctor_Analysis

Out[110]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top;
} .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>DoctorName</th>
<th>Specialization</th> <th>TotalAppointments</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th>
<th>3</th> <th>4</th> <th>...</th> <th>995</th> <th>996</th> <th>997</th> <th>998</th> <th>999</th>
</tbody> <tbody></tbody>



|            |                             |     |
|------------|-----------------------------|-----|
| Quintina   | Oncologist                  | 5   |
| Moyna      | Cardiology                  | 5   |
| Lorenza    | Ophthalmology               | 5   |
| Alex       | Dermatology                 | 4   |
| Karena     | Otolaryngologists           | 4   |
| ...        | ...                         | ... |
| Laure      | Radiology                   | 0   |
| Robbi      | Allergists                  | 0   |
| Anestassia | Obstetric Anesthesiologists | 0   |
| Elbertina  | Psychiatry                  | 0   |
| Vanessa    | Gastroenterology            | 0   |



<p>1000 rows × 3 columns</p>
```

```
In [ ]:

In [ ]: #Hospital management can identify:

#High-demand doctors
#Low-utilization doctors
#Staffing requirements
#Specialty demand

In [ ]: # Which specialization receives the most appointments?

In [90]: Specialization_analysis = pd.read_sql_query("""select D.Specialization,
count(A.AppointmentID) as TotalAppointments
from Cleaned_Doctor D
join Cleaned_Appointment A
ON D.DoctorID= A.DoctorID
Group by D.Specialization
order by TotalAppointments DESC""",conn)

In [91]: Specialization_analysis
```

```
Out[91]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top;
} .dataframe tthead th { text-align: right; } </style> <tthead> <th></th> <th>Specialization</th>
<th>TotalAppointments</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th>
<th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th>
<th>13</th> <th>14</th> <th>15</th> <th>16</th> <th>17</th> <th>18</th> <th>19</th> <th>20</th>
<th>21</th> <th>22</th> <th>23</th> </tbody> <tbody></tbody>

Ophthalmology 61
Otolaryngologists 57
Allergists 56
Gastroenterology 55
Surgery 54
Internists 52
Anesthesiology 52
Radiology 50
Pediatrics 50
Emergency Medicine 50
Oncologist 49
Infectious disease 48
Pulmonologists 47
Geriatrician 45
Obstetric Anesthesiologists 44
Neurology 43
Dermatology 43
Endocrinologist 40
Critical Care Medicine 39
Cardiology 37
Psychiatry 35
Family Medicine 29
Nephrology 28
Hospice and Palliative Care 27

In [ ]:

In [ ]: #If Cardiology receives much higher demand than another specialty, the hospital can consider:

#Additional doctors
#More appointment slots
#Better scheduling
#Additional resources

In [ ]:

In [98]: # Cleaning the Billing data

In [ ]: billings['InvoiceID'] = pd.to_numeric(billings['InvoiceID'], errors='coerce')
billings['Amount'] = pd.to_numeric(billings['Amount'], errors='coerce')
billings = billings.dropna(subset=['InvoiceID', 'Amount'])

In [85]: billings

Out[85]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top;
} .dataframe tthead th { text-align: right; } </style> <tthead> <th></th> <th>InvoiceID</th> <th>PatientID</th>
<th>Items</th> <th>Amount</th> </thead> <tbody> </tbody> <tbody></tbody>

In [ ]: # Displaying cleaned data (just to see)

In [86]: print("\nbillings")
print(billings.head())
```

```
billings
Empty DataFrame
Columns: [InvoiceID, PatientID, Items, Amount]
Index: []
```

```
In [41]: # Which patients generate the highest billing?
```

```
# This can help understand
# High value patients
# Revenue Concentration
# Patient spending patterns
# Billing trend
# In a real_hospital this should be handled carefully because patient billing information is sensitive.
```

```
In [100]: billing_analysis= pd.read_sql_query(""" select P.PatientID,
concat(P.FirstName,' ',P.LastName) as PatientName,
sum(B.Amount) AS total_billing
from Cleaned_Patient P
join Billing B
On P.PatientID = B.PatientID
group by
P.PatientID,
P.FirstName,
P.LastName
order by total_billing DESC""",conn)
```

```
In [101]: billing_analysis
```

```
Out[101]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top;
} .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>PatientID</th> <th>PatientName</th>
<th>total_billing</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>...
</th> <th>681</th> <th>682</th> <th>683</th> <th>684</th> <th>685</th> </tbody> <tbody></tbody>
```

733	Glenda LaRue	4035149
-----	--------------	---------

733	Lesly Arquit	4035149
-----	--------------	---------

368	Robinia Cecile	3436703
-----	----------------	---------

368	Stephanie Agle	3436703
-----	----------------	---------

779	Candi Shaver	3283643
-----	--------------	---------

...	...	...
-----	-----	-----

921	Linnet Kirstin	16767
-----	----------------	-------

827	Charlena Breed	16567
-----	----------------	-------

827	Mureil Oriana	16567
-----	---------------	-------

403	Ericka Ortrude	15151
-----	----------------	-------

403	Lita Swanhildas	15151
-----	-----------------	-------

<p>686 rows × 3 columns</p>

```
In [ ]: # Which doctors are associated with the highest billing?
#This is a more interesting analysis because billing doesn't directly contain DoctorID.
```

```
In [ ]: #We need:
#Patient → Appointment → Doctor and Patient → Billing
```

```
In [88]: Total_Analysis= pd.read_sql_query(""" select D.DoctorName,D.Specialization,sum(B.Amount) as Total_Billing
from Cleaned_Appointment A
join Cleaned_Doctor D
on A.DoctorID=D.DoctorID
join Billing B
on A.PatientID=B.PatientID
Group by D.DoctorID,D.DoctorName,D.Specialization
ORDER BY Total_Billing DESC """,conn)
```

```
In [89]: Total_Analysis
```

```
Out[89]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top;
} .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>DoctorName</th>
<th>Specialization</th> <th>Total_Billing</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th>
<th>3</th> <th>4</th> <th>...</th> <th>515</th> <th>516</th> <th>517</th> <th>518</th> <th>519</th>
</tbody> <tbody></tbody>

Gloria                Geriatrician  5846564
Adriana                Ophthalmology 5603885
Mariele                Ophthalmology 5603885
Leanna                 Anesthesiology 5401080
Jean                  Ophthalmology 4984876
...                   ...          ...
Joeann                 Radiology   16790
Molli  Hospice and Palliative Care  16790
Berta                  Cardiology   16567
Marita                 Surgery       16567
Cyndie                 Dermatology  15151

<p>520 rows × 3 columns</p>
```

```
In [ ]: #Billing did not directly contain DoctorID, so I used the relationship between PatientID and
#Appointment to associate billing information with doctors.
```

```
In [ ]: #Cleaning Appointment data
```

```
In [96]: Appointment['AppointmentID'] = pd.to_numeric(Appointment['AppointmentID'], errors='coerce')
Appointment['PatientID'] = pd.to_numeric(Appointment['PatientID'], errors='coerce')
Appointment['DoctorID'] = pd.to_numeric(Appointment['DoctorID'], errors='coerce')
Appointment = Appointment.dropna(subset=['AppointmentID', 'PatientID', 'DoctorID'])
```

```
In [98]: print("\nAppointment Data")
print(Appointment.head())

Appointment Data
AppointmentID      Date      Time  PatientID  DoctorID
0           639  2022-04-08  2023-12-23T14:33:46.406Z      109      462
1           404  2023-04-10  2023-12-23T14:33:46.407Z      823      774
2           281  2022-12-09  2023-12-23T14:33:46.407Z      465      226
3           628  2020-01-08  2023-12-23T14:33:46.407Z      443      656
4           889  2023-08-30  2023-12-23T14:33:46.407Z      852      262
```

```
In [99]: #. Analysis 4 – Appointment Trend
#This is especially important if you're talking about real-time/near-real-time monitoring.
```

```
In [ ]: #What does this tell management?
#We can identify:
#increasing patient demand
#low-demand days
#peak periods
#unusual spikes
#changes in appointment volume
```

```
In [ ]: #Hospital management can use this to:

#Allocate doctors
#Plan staff
#Manage appointment slots
#Reduce waiting time
#Improve resource utilization
```

```
In [ ]: # Appointment Time Analysis
```

```
In [95]: Time_analysis = pd.read_sql_query("""select Time, count(AppointmentID) as TotalAppointment
from Cleaned_Appointment
Group by Time
order by TotalAppointment DESC""",conn)
```

```
In [96]: Time_analysis
```

```
Out[96]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe tthead th { text-align: right; } </style> <thead> <th></th> <th>Time</th> <th>TotalAppointment</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> </tbody> <tbody></tbody>

2023-12-23T14:33:46.415Z    171
2023-12-23T14:33:46.410Z    134
2023-12-23T14:33:46.411Z    125
2023-12-23T14:33:46.414Z    117
2023-12-23T14:33:46.417Z     84
2023-12-23T14:33:46.416Z     82
2023-12-23T14:33:46.408Z     82
2023-12-23T14:33:46.413Z     73
2023-12-23T14:33:46.412Z     63
2023-12-23T14:33:46.407Z     61
2023-12-23T14:33:46.409Z      7
2023-12-23T14:33:46.406Z      1
```

```
In [94]: Appointment_Trend=pd.read_sql_query("""select Date,Count(AppointmentID) as Total_Appointment
from Cleaned_Appointment
Group by Date
order by Total_Appointment desc""",conn)
```

```
In [95]: Appointment_Trend
```

```
Out[95]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe tthead th { text-align: right; } </style> <thead> <th></th> <th>Date</th> <th>Total_Appointment</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>...</th> <th>716</th> <th>717</th> <th>718</th> <th>719</th> <th>720</th> </tbody> <tbody></tbody>

2023-11-19      4
2023-11-10      4
2023-04-29      4
2022-06-24      4
2021-10-15      4
...
2020-01-14      1
2020-01-07      1
2020-01-05      1
2020-01-04      1
2020-01-01      1

<p>721 rows × 2 columns</p>
```

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```